Programming Principles I

Week 3 · Class 6

Loop patterns in C

Totals and counters · checking input · break and continue · tracing

```
for (int i = 1; i <= 5; i++) {
    sum += i;
}
```

What Tuesday gave us

1. Four parts: **start**, **check**, **body**, **update**
2. `while (count <= 3) { ... }` repeats while the check is true
3. `for (int i = 1; i <= 5; i++)` makes `i` go 1, 2, 3, 4, 5
4. `i++` means `i = i + 1`
5. `sum += i` means `sum = sum + i`
6. A loop that never stops: **Ctrl+C**

Recap: what is on the screen?

PREDICT

What does this loop print?

```
for (int i = 5; i >= 1; i--) {
    printf("%d ", i);
}
```

Recap: what is on the screen?

PREDICT

What does this loop print?

```
for (int i = 5; i >= 1; i--) {
    printf("%d ", i);
}
```

ANSWER

```
5 4 3 2 1
```

Today's roadmap

1. Totals and counters
2. Checking input with `while`
3. Where a loop starts and stops
4. `break` and `continue`
5. Worked examples and common bugs
6. Tracing and practice

> Tuesday: how a loop is **written**. Today: what loops are **used for**.

1 Accumulator pattern

Building a total, one round at a time

Accumulator pattern: building a total

Add 1 + 2 + 3 + 4 + 5. Now add 1 to 1000.

You need a loop.

And you need **one box** that remembers the total so far.

Think of the till in a shop. It starts at zero. Every item adds to it. At the end you read the total.

The whole program: sum.c

```c
#include <stdio.h>

int main(void) {
    int sum = 0;

    for (int i = 1; i <= 5; i++) {
        sum += i;
        printf("after %d: sum = %d\n", i, sum);
    }

    printf("Sum = %d\n", sum);
    return 0;
}
```

Before the loop: start at zero

```c
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
    printf("after %d: sum = %d\n", i, sum);
}

printf("Sum = %d\n", sum);
```

Before the loop.

Make the box. Put 0 in it.

Inside the loop: add to it

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
    printf("after %d: sum = %d\n", i, sum);
}

printf("Sum = %d\n", sum);
```

Inside the loop.

`sum += i` means `sum = sum + i`

After the loop: use it

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
    printf("after %d: sum = %d\n", i, sum);
}

printf("Sum = %d\n", sum);
```

After the loop.

Now the total is ready.

Round 1: the total grows

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
}

printf("Sum = %d\n", sum);
```

round	i	sum before	sum after
1	1	0	1

Round 1: `sum` is 0. Add `i`, which is 1. Now `sum` is 1.

Round 2: the total remembers

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
}

printf("Sum = %d\n", sum);
```

round	i	sum before	sum after
1	1	0	1
2	**2**	**1**	**3**

Round 2: `sum` is still 1 from the last round. Add 2. Now `sum` is 3.

Round 5: the total is ready

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
}

printf("Sum = %d\n", sum);
```

round	i	sum before	sum after
1	1	0	1
2	2	1	3
3	3	3	6
4	4	6	10
5	**5**	**10**	**15**

After round 5 the check `6 <= 5` is false. The loop ends. `Sum = 15`.

Run it

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
    printf("after %d: sum = %d\n", i, sum);
}

printf("Sum = %d\n", sum);
```

Output

```
after 1: sum = 1
after 2: sum = 3
after 3: sum = 6
after 4: sum = 10
after 5: sum = 15
Sum = 15
```

The pattern has three places

Before	Inside	After
Make the total. Set it to zero. `int sum = 0;`	Add to it, every round. `sum += i;`	Use it. Print it. `printf(...)`

You will write this pattern a hundred times. Learn the three places by heart.

What if sum starts from 100?

PREDICT

Change one line:

```
int sum = 100;
```

What is the last line on the screen?

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
}

printf("Sum = %d\n", sum);
```

What if sum starts from 100?

PREDICT

Change one line:

`int sum = 100;`

What is the last line on the screen?

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum += i;
}

printf("Sum = %d\n", sum);
```

ANSWER

`Sum = 115`

The loop still adds 15.

Common mistake: resetting inside the loop

CAREFUL

One new line inside the loop:

```
sum = 0;
```

What does it print now?

```
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum = 0;
    sum += i;
}

printf("Sum = %d\n", sum);
```

Common mistake: the total is lost

```c
int sum = 0;

for (int i = 1; i <= 5; i++) {
    sum = 0;
    sum += i;
}

printf("Sum = %d\n", sum);
```

Output

```
Sum = 5
```

Every round, `sum` goes back to 0.

The till forgets everything.

No error. No warning. Wrong answer.

Common mistake: making sum inside the loop

```
for (int i = 1; i <= 5; i++) {
    int sum = 0;
    sum += i;
}

printf("Sum = %d\n", sum);
```

gcc says

```
error: 'sum' undeclared
```

A box made inside { } is gone after the }.

Rule: make the total and set it to zero **before** the loop.

More totals: you choose where it stops

```c
int n;
int sum = 0;

printf("Add the numbers from 1 to: ");
scanf("%d", &n);

for (int i = 1; i <= n; i++) {
    sum += i;
}
printf("Sum = %d\n", sum);
```

Output

```
Add the numbers from 1 to: 100
Sum = 5050
```

Same pattern. The loop stops at n.

More totals: add until the user types 0

```c
int value;
int sum = 0;

printf("Type numbers. Type 0 to stop.\n");
scanf("%d", &value);

while (value != 0) {
    sum += value;
    scanf("%d", &value);
}
printf("Sum = %d\n", sum);
```

Output

```
Type numbers. Type 0 to stop.
5 10 -3 8 0
Sum = 20
```

How many numbers? We do not know. So: `while`.

2 Counter pattern

Add one, but only when something happens

Counter pattern: counting events

> **A counter grows by one when something happens.**

How many students passed?

How many payments failed?

How many of these numbers are positive?

The program: count.c

```
int count = 0;
int n, value;

printf("How many values? ");
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    if (value > 0) {
        count++;
    }
}
printf("Positive values: %d\n", count);
```

A counter starts at 0

```c
int count = 0;
int n, value;

printf("How many values? ");
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    if (value > 0) {
        count++;
    }
}
printf("Positive values: %d\n", count);
```

Nothing is counted yet.

Before the loop, like the total.

An if inside a loop

```
int count = 0;
int n, value;

printf("How many values? ");
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    if (value > 0) {
        count++;
    }
}
printf("Positive values: %d\n", count);
```

The question is asked again **every round**.

`count++` only when the answer is yes.

How many positive values?

PREDICT

You type `5`, then:

```
3 -1 0 7 -4
```

What does it print?

```
for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    if (value > 0) {
        count++;
    }
}
printf("Positive values: %d\n", count);
```

How many positive values?

```
for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    if (value > 0) {
        count++;
    }
}
```

round	value	value > 0?	count
1	3	yes	1
2	-1	no	1
3	0	no	1
4	7	yes	2
5	-4	no	2

`Positive values:` 2. Zero is **not** positive: `0 > 0` is false.

Count without input

PREDICT

How many numbers from 1 to 20 can be divided by 3?

What does it print?

```
int count = 0;

for (int i = 1; i <= 20; i++) {
    if (i % 3 == 0) {
        count++;
    }
}

printf("Count: %d\n", count);
```

Count without input

PREDICT

How many numbers from 1 to 20 can be divided by 3?

What does it print?

```
int count = 0;

for (int i = 1; i <= 20; i++) {
    if (i % 3 == 0) {
        count++;
    }
}

printf("Count: %d\n", count);
```

ANSWER

```
Count: 6
```

3, 6, 9, 12, 15, 18

A cousin: the largest value so far

```
for (int i = 1; i < n; i++) {
    scanf("%d", &value);
    if (value > largest) {
        largest = value;
    }
}
printf("Largest: %d\n", largest);
```

Output

```
How many values? 5
3 9 -2 14 6
Largest: 14
```

One box remembers the **largest so far**.

It starts as the first value typed.

3

Checking input with `while`

Ask again until the input is good

Input validation with while

Users type nonsense. A good program asks again.

An age of `-5`. An age of `200`.

How many times must we ask? **We do not know.**

When you do not know how many rounds you need: `while`.

The program: age.c

```
int age;

printf("Enter age: ");
scanf("%d", &age);

while (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}

printf("Accepted: %d\n", age);
return 0;
```

Step 1: ask once, before the loop

```
int age;

printf("Enter age: ");
scanf("%d", &age);

while (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}

printf("Accepted: %d\n", age);
return 0;
```

Ask. Read one value.

Step 2: is the input bad?

```
int age;

printf("Enter age: ");
scanf("%d", &age);

while (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}

printf("Accepted: %d\n", age);
return 0;
```

below 0 **or** above 120

The condition describes the **bad** input.

Step 3: ask again

```
int age;

printf("Enter age: ");
scanf("%d", &age);

while (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}

printf("Accepted: %d\n", age);
return 0;
```

This `scanf` is the **update**.

It is the only line that can make the condition false.

Step 4: after the loop, trust the input

```
int age;

printf("Enter age: ");
scanf("%d", &age);

while (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}

printf("Accepted: %d\n", age);
return 0;
```

The loop has ended.

So the age is good. Every line from here can trust it.

Run it

```
int age;

printf("Enter age: ");
scanf("%d", &age);

while (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}

printf("Accepted: %d\n", age);
return 0;
```

Output

```
Enter age: -5
Invalid. Try again: 200
Invalid. Try again: 19
Accepted: 19
```

Test the edges

You type	What happens	Why
`0`	accepted at once	`0 < 0` is false
`120`	accepted at once	`120 > 120` is false
`121`	Invalid. Try again:	`121 > 120` is true
`-1`	Invalid. Try again:	`-1 < 0` is true

For `0` and `120` the body runs **zero times**. A `while` checks first.

Same shape, new condition

age: 0 to 120

```
while (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}
```

month: 1 to 12

```
while (month < 1 || month > 12) {
    printf("Invalid. Try again: ");
    scanf("%d", &month);
}
```

Only the condition changes. It always describes the **bad** input.

4 Choosing loop boundaries

< or <=? Start at 0 or at 1?

Choosing loop boundaries

0 to n - 1, with <

```
for (int i = 0; i < 5; i++) {
    printf("%d ", i);
}
// 0 1 2 3 4
```

1 to n, with <=

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
// 1 2 3 4 5
```

Both run **five** times. Both are correct. The left style comes back with arrays.

Which loop prints exactly 10 numbers?

1
```
for (int i = 0; i < 10; i++)
```

2
```
for (int i = 1; i <= 10; i++)
```

3
```
for (int i = 0; i <= 10; i++)
```

4
```
for (int i = 1; i < 10; i++)
```

Hands up for each one. More than one can be right.

Which loop prints exactly 10 numbers?

Loop head	Prints	How many
`i = 0; i < 10`	0 to 9	10
`i = 1; i <= 10`	1 to 10	10
`i = 0; i <= 10`	0 to 10	**11**
`i = 1; i < 10`	1 to 9	**9**

One too many, or one too few: the **off-by-one error**. The most common loop bug there is.

Do not choose < or <= by feeling

> **Ask two questions. Then test the last value by hand.**

1. What is the **first** value I want?
2. What is the **last** value I want?

I want 10 included. Is `10 <= 10` true? Yes. Is `10 < 10` true? No: that loop stops at 9.

5 `break` and `continue`

Two words that change the normal flow of a loop

break: what prints?

PREDICT

`i` goes from 1 to 10.

But look at the `if`.

What is on the screen?

```
for (int i = 1; i <= 10; i++) {
    if (i == 6) {
        break;
    }
    printf("%d ", i);
}
```

break: leave the loop now

```
for (int i = 1; i <= 10; i++) {
    if (i == 6) {
        break;
    }
    printf("%d ", i);
}
```

Output

```
1 2 3 4 5
```

At 6 the loop ends. **Completely.**

6 is not printed. 7 to 10 never happen.

continue: what prints?

PREDICT

`i` goes from 1 to 5.

What is on the screen?

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    printf("%d ", i);
}
```

continue: skip the rest of this round

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    printf("%d ", i);
}
```

Output

```
1 2 4 5
```

At 3 the `printf` is skipped.

Then the loop goes on with 4. The update `i++` still runs.

break and continue

break	continue
Out of the loop. For good.	Skip to the next round.

Use them carefully. Most simple loops need neither: the condition does the work.

Two more examples

break: squares up to 30

```
for (int i = 1; i <= 10; i++) {
    if (i * i > 30) {
        break;
    }
    printf("%d ", i * i);
}
```

continue: only odd numbers

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue;
    }
    printf("%d ", i);
}
```

Left prints 1 4 9 16 25. Right prints 1 3 5 7 9.

A real use of break: three tries for a PIN

```
int ok = 0;           // 0: not right yet

for (int tries = 1; tries <= 3; tries++) {
    printf("PIN (try %d of 3): ", tries);
    scanf("%d", &pin);
    if (pin == 1234) {
        ok = 1;
        break;        // right: stop asking
    }
}
```

Right PIN: no more tries are needed, so break. After the loop, ok says why we left.

6 Worked examples

Two whole programs with a loop inside

Worked example: multiplication table

```c
#include <stdio.h>

int main(void) {
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }
    return 0;
}
```

Same code, different value

```
for (int i = 1; i <= 10; i++) {
    printf("%d x %d = %d\n", n, i, n * i);
}
```

Output

```
Enter a number: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

One line of code, ten lines on the screen

`n` stays the same. `i` changes every round.

Three holes: `%d x %d = %d`

Three values: `n`, `i`, `n * i`

Question: I want the table up to 12. What do I change?

A look ahead: a loop inside a loop

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 5; col++) {
        printf("%4d", row * col);
    }
    printf("\n");
}
```

Output

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
```

For **every** row, the inner loop runs 5 times.

Not for Quiz 2. It comes back in October.

Worked example: factorial

> **$5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$**

It is the running total again.

But with **times**, not plus: a running **product**.

Worked example: factorial

```c
#include <stdio.h>

int main(void) {
    int n = 5;
    int fact = 1;

    for (int i = 1; i <= n; i++) {
        fact *= i;
    }

    printf("%d\n", fact);
    return 0;
}
```

Factorial, round by round

```
int n = 5;
int fact = 1;

for (int i = 1; i <= n; i++) {
    fact *= i;
}

printf("%d\n", fact);
```

i	fact before	fact *= i	fact after
1	1	1 × 1	1
2	**1**	**1 × 2**	**2**

Round 2: `fact` is 1. Times 2. Now `fact` is 2.

Factorial, round by round

```
int n = 5;
int fact = 1;

for (int i = 1; i <= n; i++) {
    fact *= i;
}

printf("%d\n", fact);
```

i	fact before	fact *= i	fact after
1	1	1 × 1	1
2	1	1 × 2	2
3	2	2 × 3	6
4	6	6 × 4	24
5	**24**	**24 × 5**	**120**

After round 5: `fact` is 120. The program prints `120`.

Why does fact start at 1?

QUESTION

The sum started at 0.

Why not `int fact = 0;` ?

```
int n = 5;
int fact = 0;

for (int i = 1; i <= n; i++) {
    fact *= i;
}

printf("%d\n", fact);
```

If fact starts at 0

```
int n = 5;
int fact = 0;

for (int i = 1; i <= n; i++) {
    fact *= i;
}

printf("%d\n", fact);
```

Output

```
0
```

Anything times 0 is 0.

`fact` stays 0 for ever.

When you **add**, start at 0. When you **multiply**, start at 1.

Mini-task: read n from the user

```
int n;
int fact = 1;

printf("Enter n: ");
scanf("%d", &n);

for (int i = 1; i <= n; i++) {
    fact *= i;
}
printf("%d! = %d\n", n, fact);
```

Two lines change.

`int n;` and a `scanf`.

Now watch the numbers grow

You type	Screen shows	Correct?
5	5! = 120	yes
10	10! = 3628800	yes
12	12! = 479001600	yes
13	13! = 1932053504	no
17	17! = -288522240	no

An `int` box has a size: the largest number is 2147483647. 13! does not fit. **No error. No warning.**

7

Common Week 3 bugs

Small-looking, but they change the whole result

Common Week 3 bugs

1. **Missing update**: the loop never stops
2. **Wrong comparison**: `<` against `<=`
3. **Resetting inside the loop**: the total is lost
4. **Semicolon after the loop**: an empty loop
5. `=` instead of `==`: an assignment, not a question
6. **Bad starting value**: `i` or `fact` starts wrong

The semicolon bug

CAREFUL

One character too many.

What do you expect on the screen?

```
int i;

for (i = 1; i <= 5; i++);
{
    printf("%d\n", i);
}
```

The semicolon bug: it prints 6, once

```
int i;

for (i = 1; i <= 5; i++);
{
    printf("%d\n", i);
}
```

Output

```
6
```

The `;` **is** the body: do nothing, five times.

Then `i` is 6. The block runs once.

gcc -Wall warns: `this 'for' clause does not guard...` Read your warnings.

= instead of == inside a loop

```
for (int i = 1; i <= 5; i++) {
    if (i = 3) {
        printf("three\n");
    }
}
```

`i = 3` puts 3 into `i`. **Every round.**

`i` never gets past 3. The loop never ends.

Ctrl+C. In a condition: always ==.

Two loops that never stop

Missing update

```
int i = 1;

while (i <= 5) {
    printf("%d\n", i);
}
```

Semicolon after while

```
int x = 1;

while (x <= 5);
{
    printf("%d\n", x);
    x++;
}
```

Left: prints 1 for ever, and gcc says nothing. Right: prints nothing and hangs.
Both: **Ctrl+C**.

The debugging habit

Start	Check	Body	Update
Where does it begin?	When does it stop?	What is repeated?	What changes each round?

> Before you run a loop, put your finger on each of the four parts. If you cannot find one, that is your bug.

8

Code tracing exercise

Laptops down. Pen and paper. Trace first, then run.

Code tracing exercise

YOUR TURN

1. x and y after each round?
2. How many rounds?
3. What is printed?
4. Which line moves the loop toward stopping?

```
int x = 2;
int y = 0;

while (x < 8) {
    y += x;
    x += 2;
}

printf("x=%d y=%d\n", x, y);
```

Tracing: round 1

```
int x = 2;
int y = 0;

while (x < 8) {
    y += x;
    x += 2;
}

printf("x=%d y=%d\n", x, y);
```

round	x < 8?	y	x
1	true	2	4

2 < 8 is true. y is 0 + 2 = 2. Then x is 2 + 2 = 4.

Tracing: round 2

```
int x = 2;
int y = 0;

while (x < 8) {
    y += x;
    x += 2;
}

printf("x=%d y=%d\n", x, y);
```

round	x < 8?	y	x
1	true	2	4
2	**true**	**6**	**6**

4 < 8 is true. y is 2 + 4 = 6. Then x is 6.

Tracing: round 3

```
int x = 2;
int y = 0;

while (x < 8) {
    y += x;
    x += 2;
}

printf("x=%d y=%d\n", x, y);
```

round	x < 8?	y	x
1	true	2	4
2	true	6	6
3	**true**	**12**	**8**

6 < 8 is true: the loop does not "feel" that 6 is close to 8. y is 12. x is 8.

Tracing: the loop ends

```
int x = 2;
int y = 0;

while (x < 8) {
    y += x;
    x += 2;
}

printf("x=%d y=%d\n", x, y);
```

round	x < 8?	y	x
1	true	2	4
2	true	6	6
3	true	12	8
4	**false**		

`8 < 8` is **false**. Three rounds. Prints `x=8 y=12`. After a loop, x holds the first value that failed the check.

One more to trace at home

YOUR TURN

Trace it on paper.

Remember: `5 / 2` is 2.

Then run `30_trace_halving.c` to check.

```
int n = 10;
int steps = 0;

while (n > 1) {
    n = n / 2;
    steps++;
}

printf("n=%d steps=%d\n", n, steps);
```

9

Practice

Now you write

In-class practice tasks

1. Print the numbers from 1 to n
2. **Print the even numbers from 2 to n**
3. Read n values and calculate their sum
4. Read n values and count the positives
5. **Keep asking for a PIN until it is correct**
6. Print a multiplication table for one number

> Before you type: write the input, the output, the start, the stop and the update in plain words.

Task 2: even numbers from 2 to n

YOUR TURN

Read `n`.

Print the even numbers from 2 to `n`.

You type `10`: the screen shows `2 4 6 8 10`.

You type `9`: the screen shows `2 4 6 8`.

Task 2: two good ways

Jump by two

```
for (int i = 2; i <= n; i += 2) {
    printf("%d ", i);
}
```

Visit every number, ask

```
for (int i = 1; i <= n; i++) {
    if (i % 2 == 0) {
        printf("%d ", i);
    }
}
```

Both are right. `i % 2 == 0` means: the remainder after dividing by 2 is 0, so `i` is even.

Task 5: keep asking for the PIN

YOUR TURN

The right PIN is `1234`.

Wrong PIN: ask again.

Right PIN: print `Welcome`.

```
PIN: 1111
Wrong. Try again: 4321
Wrong. Try again: 1234
Welcome
```

Task 5: it is the age check again

```
int pin;

printf("PIN: ");
scanf("%d", &pin);

while (pin != 1234) {
    printf("Wrong. Try again: ");
    scanf("%d", &pin);
}
```

`pin != 1234`: the PIN is **not** 1234.

That is the bad input. So the loop goes on.

Quiz-style question 1

PREDICT

What is printed?

```
int i = 1;
while (i <= 3) {
    printf("%d ", i);
    i++;
}
```

Quiz-style question 2

PREDICT

What is the output?

```
int x = 5;
printf("%d ", x++);
printf("%d\n", x);
```

Quiz-style question 2: the answer

PREDICT

What is the output?

```
int x = 5;
printf("%d ", x++);
printf("%d\n", x);
```

ANSWER

```
5 6
```

x++: old value first.

Quiz-style question 3

PREDICT

What is the final value of `sum`?

Show the trace, not only the answer.

```
int sum = 0;
for (int i = 1; i <= 4; i++) {
    sum += i;
}
```

Week 3 summary

1. `while`: repeat while a condition is true
2. `for`: the compact loop with a counter
3. `++`, `--`, `+=` and friends: short updates
4. Accumulator: a total starts at 0, a product at 1
5. Counter: `count++` inside an `if` inside the loop
6. Tracing: a table of values beats guessing

Always ask, for every loop

1. Where does it **start**?
2. When does it **stop**?
3. What **changes** each time?
4. What should be **printed or stored**?

Before Tuesday

1. **Knowledge check 6** on Blackboard: today's slides
2. Tasks **1, 3, 4 and 6**: bring them, working or not
3. **Quiz 2: Fri 2 October**, on paper, classes 3 to 6
4. Today's code: Blackboard, Week 3 module, **Class 6 code**

> Next week: functions. Giving a name to a piece of program, so you can use it again.